UNIVERSIDAD TECNOLÓGICA DE LOS ANDES

FACULTAD DE INGENIERÍA

DEPARTAMENTO ACADÉMICO DE INGENIERÍAS

E.P. DE ING. DE SISTEMAS E INFORMÁTICA

Computación Gráfica

INGENIERO

Tintaya Zegarra Elvio

TAREA

"Ejercicios 3D"

ESTUDIANTE

Santos Segovia Jose Luis

CUSCO – PERÚ

2026 – I

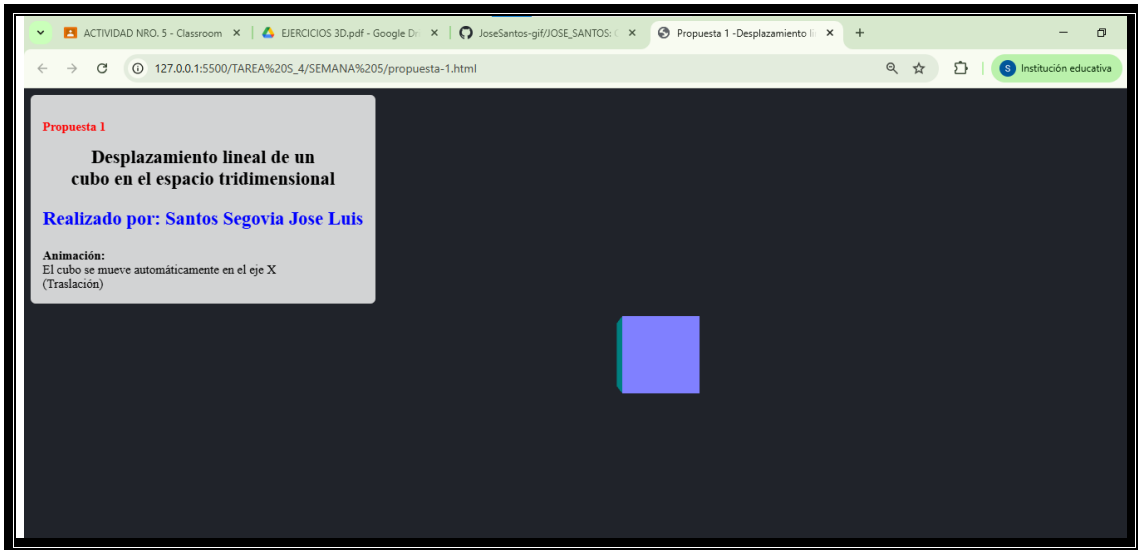
EJERCICIOS 3D

Propuesta 1: Animación de traslación de un objeto 3D en el eje X
Título de la práctica:
Desplazamiento lineal de un cubo en el espacio tridimensional
Descripción de la propuesta:
El estudiante desarrollará una escena básica en 3D utilizando JavaScript, en la cual un cubo se desplace de manera continua de izquierda a derecha sobre el eje X. La práctica estará orientada a comprender el concepto de traslación como transformación geométrica que modifica la posición de un objeto sin alterar su forma, tamaño ni orientación.
Objetivo de aprendizaje:
Comprender y aplicar la transformación de traslación en un objeto tridimensional mediante la modificación de sus coordenadas espaciales.
Acciones específicas que realizará el estudiante:
• Crear una escena 3D simple.
• Insertar un cubo como objeto principal.
• Posicionar una cámara para visualizar el objeto.
• Programar el desplazamiento horizontal automático del cubo.
• Establecer límites de recorrido para que el objeto cambie de dirección al llegar a un extremo.
Concepto central a desarrollar:
La traslación permite mover un objeto en el espacio 3D a través de los ejes X, Y o Z. En esta propuesta, el énfasis estará en el eje X para facilitar la comprensión inicial del movimiento tridimensional.
Resultado esperado:
El estudiante observará que el cubo cambia su ubicación en el espacio sin modificar su tamaño ni rotación, identificando claramente el efecto de la traslación.

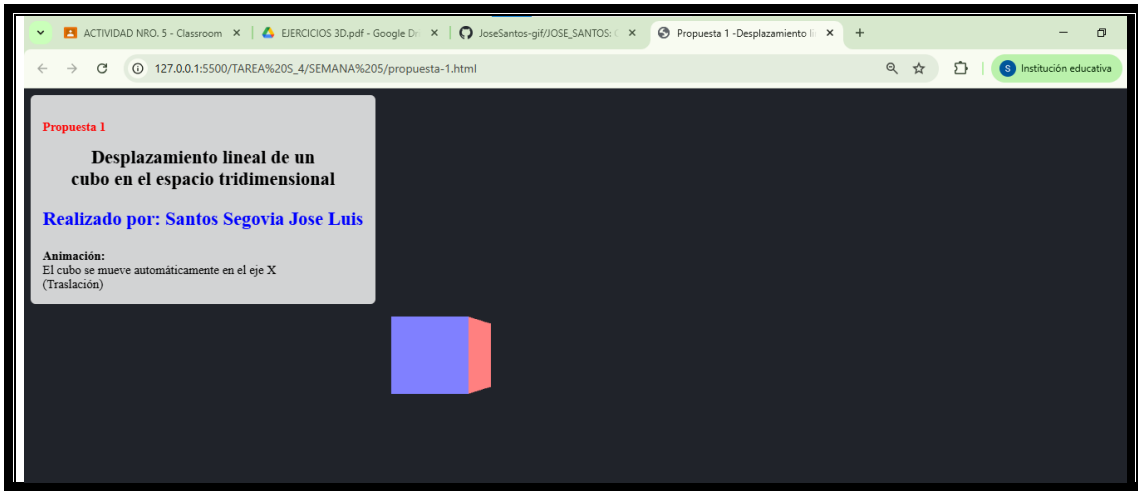
SOLUCION:

1. **Visualización en el navegador**

1.1. Lado derecho



1.2.Lado Izquierdo



2. Código

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>Propuesta 1 -Desplazamiento lineal de un cubo en el espacio
tridimensional</title>

<style>
body {
    margin: 0;
    overflow: hidden;
}

#info {
    position: absolute;
    top: 10px;
    left: 10px;
    background: rgba(255,255,255,0.8);
    padding: 20px;
    border-radius: 8px;
    font-size: 20px;
}

h1{
    font-size: 30px;
    text-align: center;
}

h3{
```

```
    font-size: 30px;
    color: blue;
}

p{
    color: red;
    font-size: 20px;;
}
</style>

</head>
<body>

<div id="info">
    <p><b>Propuesta 1</b></p>
    <h1>Desplazamiento lineal de un <br> cubo en el espacio
tridimensional</h1>
    <h3>Realizado por: Santos Segovia Jose Luis</h3>
    <b>Animación:</b><br>
    El cubo se mueve automáticamente en el eje X<br>
    (Traslación)
</div>

<!-- Importamos la librería three.js -->
<script
src="https://cdn.jsdelivr.net/npm/three@0.160.0/build/three.min.js"></s
cript>

<script>

// Creamos la escena
const scene = new THREE.Scene(); // Creamos el espacio 3D
scene.background = new THREE.Color(0x20232a); // Color de fondo oscuro

// Creamos la camara
const camera = new THREE.PerspectiveCamera(
    75, // campo de visión
    window.innerWidth / window.innerHeight, // proporción pantalla
    0.1, // distancia mínima
    1000 // distancia máxima
);

// Creamos el redendizador
const renderer = new THREE.WebGLRenderer(); // Motor gráfico
renderer.setSize(window.innerWidth, window.innerHeight); // Tamaño
pantalla
document.body.appendChild(renderer.domElement); // Agregar al HTML
```

```
// Creamos el cubo
const geometry = new THREE.BoxGeometry(1,1,1); // Geometría del cubo
const material = new THREE.MeshNormalMaterial(); // Material con
colores automáticos
const cubo = new THREE.Mesh(geometry, material); // Crear objeto
scene.add(cubo); // Añadir a la escena

// Agregamos luz
const light = new THREE.PointLight(0xffffff, 1); // Luz blanca
light.position.set(5,5,5); // Posición de la luz
scene.add(light); // Añadir a la escena

// Posicionamos la camara
camera.position.z = 5; // Alejar la cámara para ver el cubo

// Creamos las variables de traslacion
let velocidad = 0.05; // Velocidad de movimiento
let limite = 3; // Límite de desplazamiento en X

// Creamos una funcion para la animacion
function animate() {

    requestAnimationFrame(animate); // Llama continuamente la función
(loop)

    // Realizamos traslacion en 3D
    cubo.position.x += velocidad; // Mover el cubo en eje X

    // Nos permite el cambio de direccion
    if (cubo.position.x > limite || cubo.position.x < -limite) {
        velocidad *= -1; // Invierte dirección
    }

    // Nos permite redenrizar la escena
    renderer.render(scene, camera);
}

// Ejecutar animación
animate();

// ajuste responsivo
window.addEventListener('resize', () => {
    camera.aspect = window.innerWidth / window.innerHeight; //
Actualizar proporción
    camera.updateProjectionMatrix(); // Recalcular cámara
    renderer.setSize(window.innerWidth, window.innerHeight); // Ajustar
tamaño
});
```

```
</script>

</body>
</html>
```

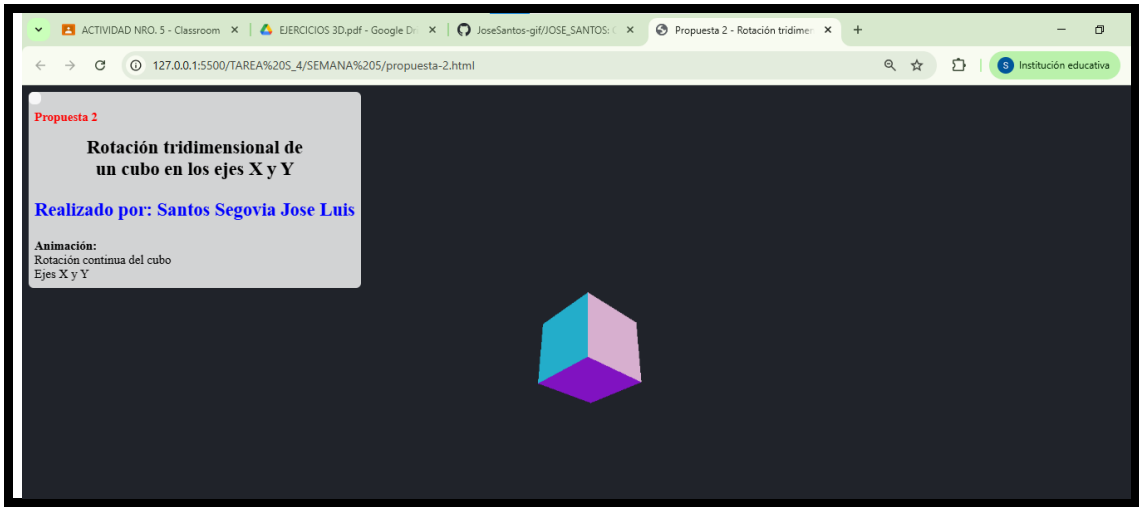
Propuesta 2: Rotación continua de un cubo sobre sus ejes
Título de la práctica:
Rotación tridimensional de un cubo en los ejes X y Y
Descripción de la propuesta:
El estudiante implementará una animación en la que un cubo rote de manera continua dentro de una escena 3D. Esta práctica permitirá comprender cómo un objeto puede cambiar su orientación en el espacio sin alterar su posición ni dimensiones. La actividad puede iniciar con rotación en un solo eje y luego ampliarse a dos ejes para mejorar la percepción espacial.
Objetivo de aprendizaje:
Aplicar la transformación de rotación sobre un objeto 3D para analizar los cambios de orientación en distintos ejes del espacio.
Acciones específicas que realizará el estudiante:
• Crear una escena tridimensional con un cubo visible.
• Configurar una cámara frontal para observar claramente la rotación.
• Programar el giro continuo del cubo.
• Probar la rotación sobre el eje X.
• Probar la rotación sobre el eje Y.
• Comparar visualmente la diferencia entre rotar en un eje y en dos ejes simultáneamente.
Concepto central a desarrollar:
La rotación modifica la orientación del objeto respecto a un eje de referencia. En

computación gráfica, esta transformación es esencial para animación, modelado, videojuegos y simulación.
Resultado esperado:
El estudiante identificará que el cubo mantiene su posición, pero cambia su orientación de forma continua, comprendiendo el comportamiento de la rotación en el espacio tridimensional.

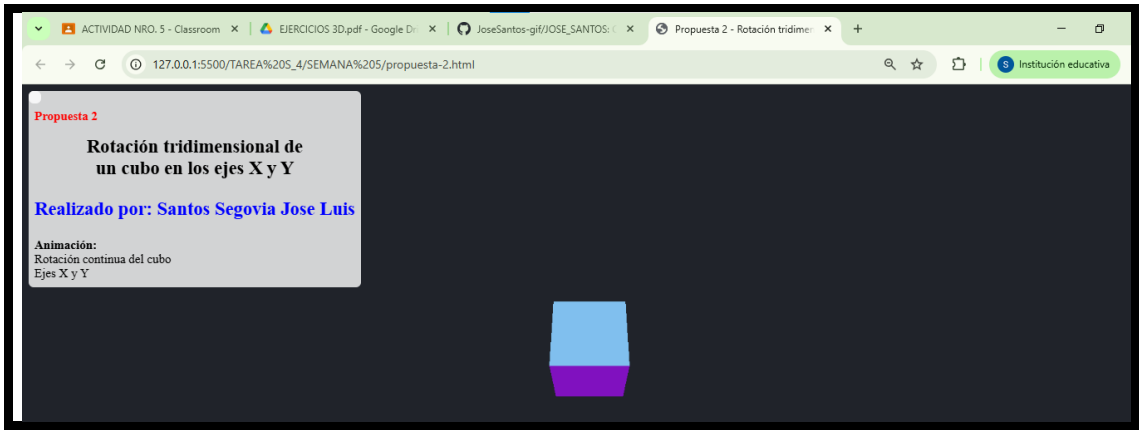
SOLUCION:

1. **Visualización en el navegador**

1.1. Rotación en el Eje X



1.2. Rotación en el eje Y



2. Código

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>Propuesta 2 - Rotación tridimensional de un cubo en los ejes
X y Y</title>

<style>
body {
    margin: 0;
    overflow: hidden;
}

#info {
    position: absolute;
    top: 10px;
```

```
    left: 10px;
    background: rgba(255,255,255,0.8);
    padding: 10px;
    border-radius: 8px;
    font-size: 20px;
}

h1{
    font-size: 30px;
    text-align: center;
}

h3{
    font-size: 30px;
    color: blue;
}

p{
    color: red;
    font-size: 20px;;
}
</style>

</head>
<body>

<div id="info">

</div>

<div id="info">
    <p><b>Propuesta 2</b></p>
    <h1>Rotación tridimensional de <br>un cubo en los ejes X y Y</h1>
    <h3>Realizado por: Santos Segovia Jose Luis</h3>
    <b>Animación:</b><br>
    Rotación continua del cubo<br>
    Ejes X y Y
</div>

<!-- Librería Three.js -->
<script
src="https://cdn.jsdelivr.net/npm/three@0.160.0/build/three.min.js"></s
cript>

<script>

// Creamos la escena
const scene = new THREE.Scene(); // Espacio 3D
```

```
scene.background = new THREE.Color(0x20232a); // Color de fondo

// Creamos la camara
const camera = new THREE.PerspectiveCamera(
    75, // campo de visión
    window.innerWidth / window.innerHeight, // proporción
    0.1, // cercano
    1000 // lejano
);

// Creamos el redendizador
const renderer = new THREE.WebGLRenderer(); // Motor gráfico
renderer.setSize(window.innerWidth, window.innerHeight); // Tamaño
document.body.appendChild(renderer.domElement); // Insertar en HTML

// Creamos el cubo
const geometry = new THREE.BoxGeometry(1,1,1); // Forma
const material = new THREE.MeshNormalMaterial(); // Material
const cubo = new THREE.Mesh(geometry, material); // Objeto
scene.add(cubo); // Agregar a la escena

// Agregamos luz
const light = new THREE.PointLight(0xffffff, 1); // Luz blanca
light.position.set(5,5,5); // Posición
scene.add(light); // Añadir

// Posicionamos la camara
camera.position.z = 5; // Alejar para ver el cubo

// Creamos las variables de rotación
let velocidadX = 0.02; // Velocidad eje X
let velocidadY = 0.03; // Velocidad eje Y

// Creamos la funcion de animación
function animate() {

    requestAnimationFrame(animate); // Loop infinito

    // Nos permite la rotacion en el eje X
    cubo.rotation.x += velocidadX; // Giro sobre eje X

    // Nos permite la rotacion en el eje Y
    cubo.rotation.y += velocidadY; // Giro sobre eje Y

    // Renderizador
    renderer.render(scene, camera);
}
```

```
// Ejecutar animación
animate();

// Ajustes responsivos
window.addEventListener('resize', () => {
    camera.aspect = window.innerWidth / window.innerHeight; // Nueva
proporción
    camera.updateProjectionMatrix(); // Actualizar
    renderer.setSize(window.innerWidth, window.innerHeight); // Ajustar
pantalla
});

</script>

</body>
</html>
```

Propuesta 3: Transformaciones combinadas en un objeto 3D controlado por el usuario
Título de la práctica:
Aplicación integrada de traslación, rotación y escalado en un objeto tridimensional
Descripción de la propuesta:
El estudiante desarrollará una escena 3D interactiva en la que podrá manipular un objeto mediante distintas transformaciones geométricas. A través del teclado, el objeto podrá desplazarse, girar y cambiar de tamaño. Esta práctica integrará los tres conceptos principales de transformación 3D y permitirá al estudiante experimentar de manera directa con el comportamiento del objeto en el espacio.
Objetivo de aprendizaje:
Integrar las transformaciones de traslación, rotación y escalado en una misma escena 3D,
controlando un objeto de forma interactiva mediante JavaScript.
Acciones específicas que realizará el estudiante:
• Crear una escena 3D con un objeto principal.
• Configurar cámara, renderizador y visualización adecuada.
• Programar teclas para mover el objeto en diferentes direcciones.
• Asignar teclas para rotar el objeto sobre uno de sus ejes.
• Asignar teclas para aumentar y reducir su tamaño.
• Verificar visualmente el efecto individual y combinado de cada transformación.
Conceptos centrales a desarrollar:
• **Traslación:** cambia la posición del objeto.
• **Rotación:** cambia la orientación del objeto.
• **Escalado:** cambia el tamaño del objeto.
Esta propuesta busca que el estudiante no solo observe las transformaciones, sino que también las controle directamente para comprender sus diferencias y aplicaciones.
Resultado esperado:
El estudiante logrará manipular un objeto 3D de manera interactiva, reconociendo con claridad cómo actúa cada transformación y cómo pueden combinarse dentro de una

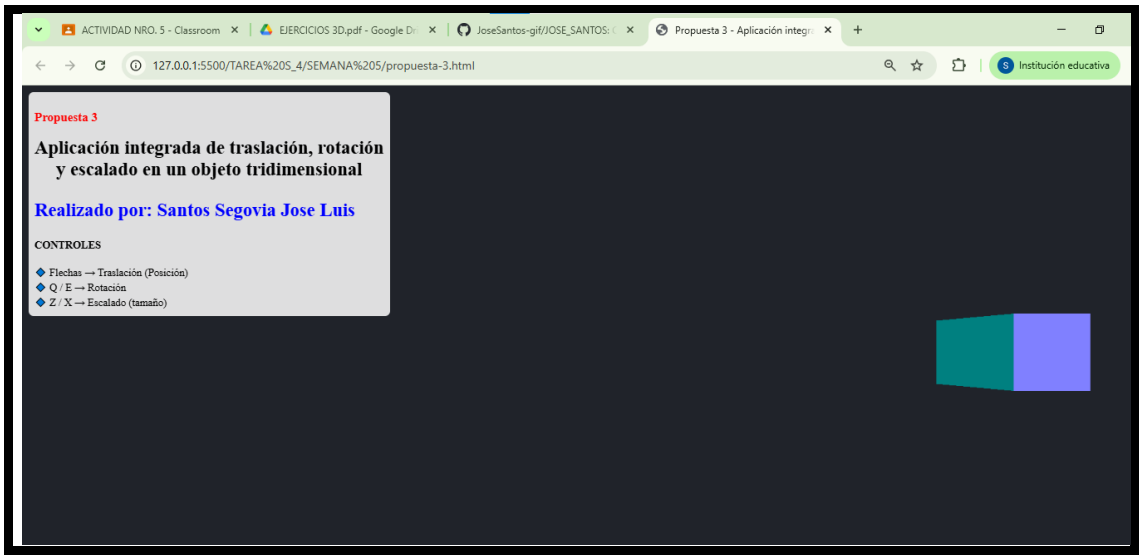
misma escena.

SOLUCION:

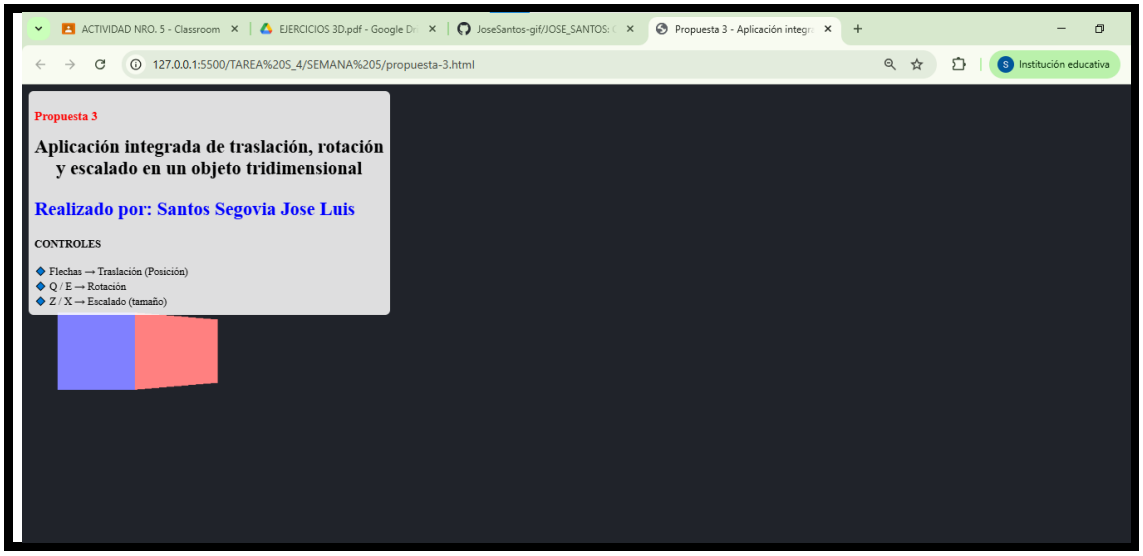
1. Visualización en el navegador

1.1. TRASLACION

1.1.1. Traslación a la derecha – Eje X

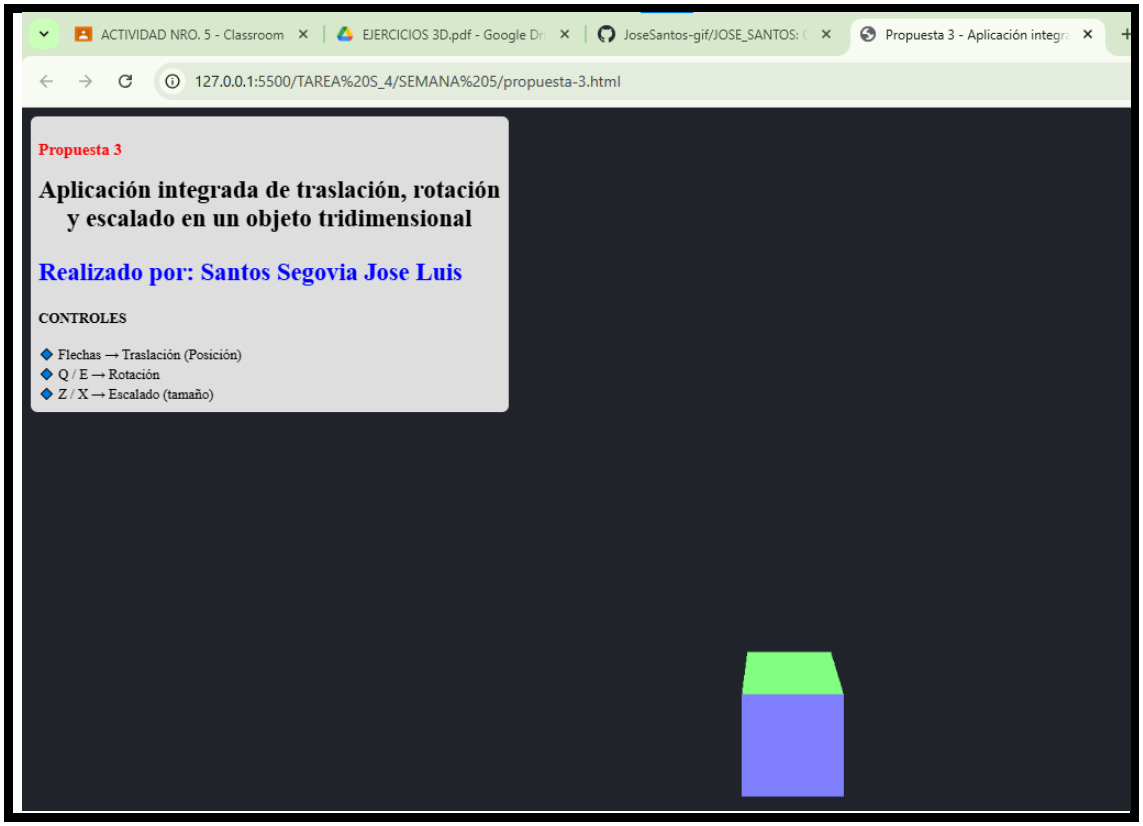


1.1.2. Traslación a la izquierda – Eje X



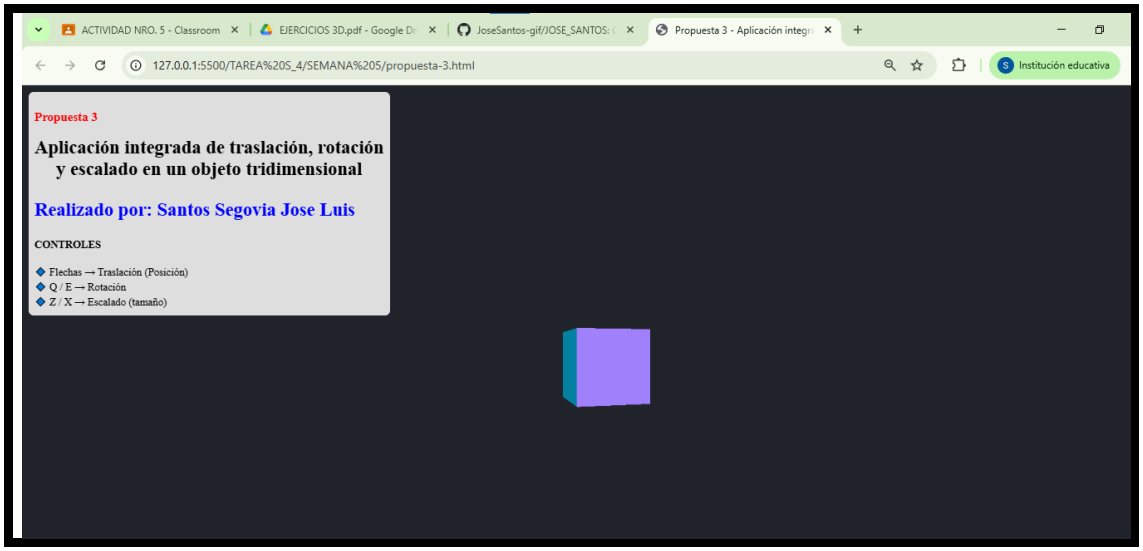
1.1.3. Traslación hacia arriba – Eje Y

1.1.4. Traslación hacia abajo – Eje Y

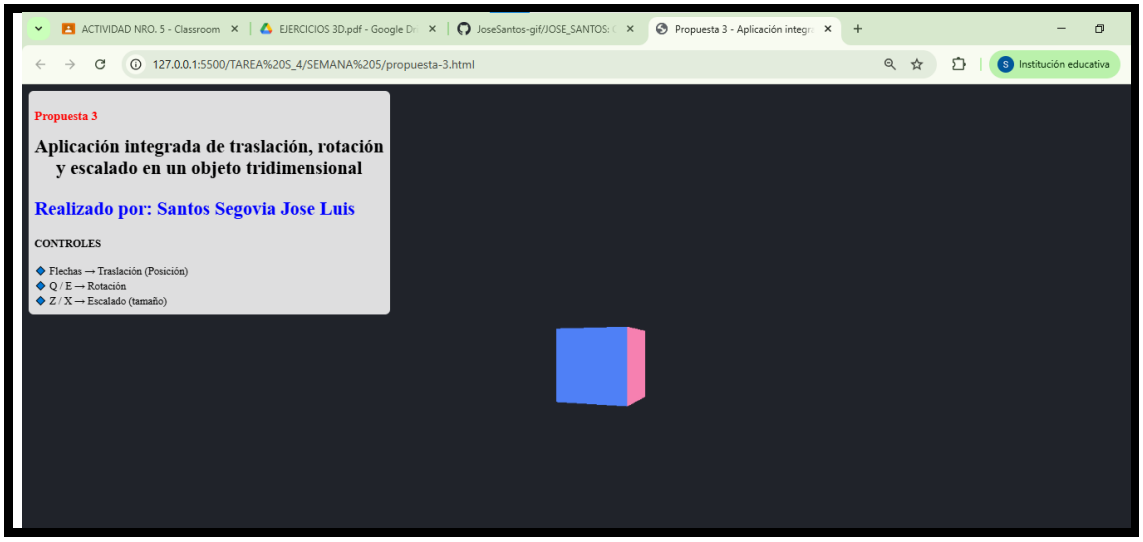


1.2.ROTACION

1.2.1. Rotación hacia el Eje X – Derecha

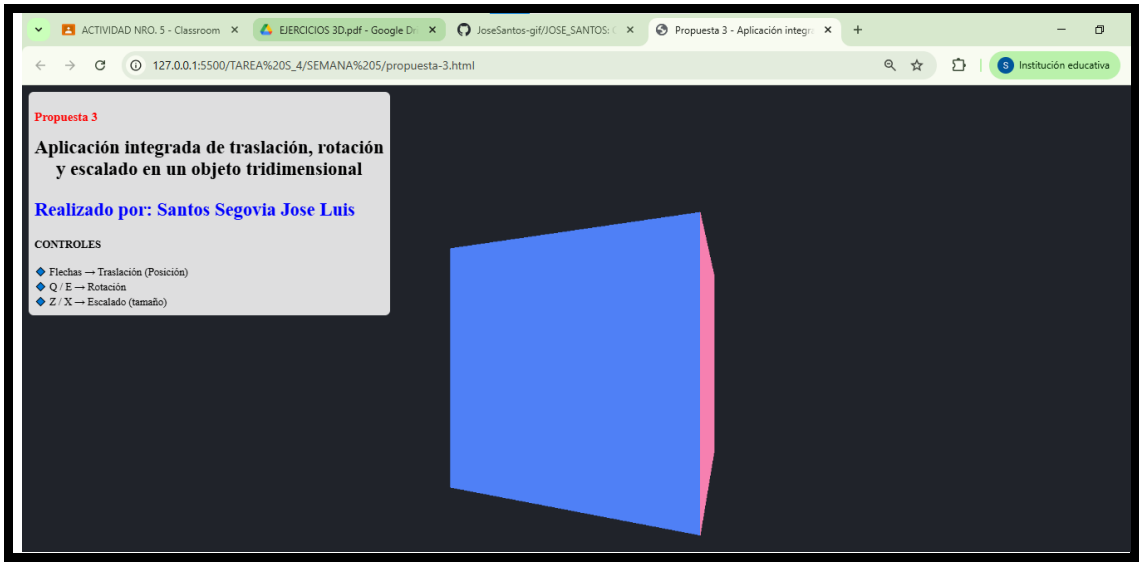


1.2.2. Rotación hacia el Eje X - Izquierda

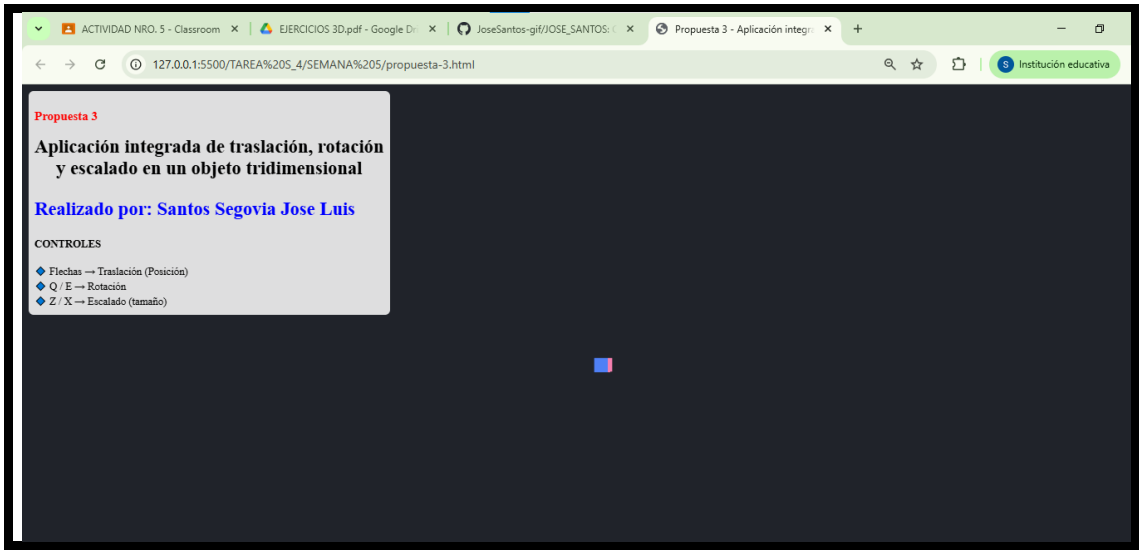


1.3.ESCALADO

1.3.1. Escalado – Agrandar tamaño



1.3.2. Escalado – Disminuir Tamaño



2. Código

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>Propuesta 3 - Aplicación integrada de traslación, rotación y
escalado en un objeto tridimensional</title>

<style>
body {
    margin: 0;
    overflow: hidden;
```

```
    font-family: Arial;
}

#info {
    position: absolute;
    top: 10px;
    left: 10px;
    background: rgba(255,255,255,0.85);
    padding: 10px;
    border-radius: 8px;
    font-size: 17px;
    font-family: 'Times New Roman', Times, serif;
}

h1{
    font-size: 30px;
    text-align: center;
}

h3{
    font-size: 30px;
    color: blue;
}

p{
    color: red;
    font-size: 20px;;
}
</style>

</head>
<body>

<div id="info">
    <p><b>Propuesta 3</b></p>
    <h1>Aplicación integrada de traslación, rotación <br>y escalado en
un objeto tridimensional</h1>
    <h3>Realizado por: Santos Segovia Jose Luis</h3>
    <b>CONTROLES</b><br><br>
    ◈ Flechas → Traslación (Posición)<br>
    ◈ Q / E → Rotación<br>
    ◈ Z / X → Escalado (tamaño)
</div>

<!-- Librería Three.js -->
<script
src="https://cdn.jsdelivr.net/npm/three@0.160.0/build/three.min.js"></s
cript>
```

```
<script>

// Creamos la escena
const scene = new THREE.Scene(); // Espacio 3D
scene.background = new THREE.Color(0x20232a); // Color de fondo

// Creamos la camara
const camera = new THREE.PerspectiveCamera(
    75, // campo de visión
    window.innerWidth / window.innerHeight, // proporción
    0.1, // cercano
    1000 // lejano
);

// Creamos el renderizador
const renderer = new THREE.WebGLRenderer(); // Motor gráfico
renderer.setSize(window.innerWidth, window.innerHeight); // Tamaño
document.body.appendChild(renderer.domElement); // Insertar en HTML

// Creamos el cubo
const geometry = new THREE.BoxGeometry(1,1,1); // Geometría
const material = new THREE.MeshNormalMaterial(); // Material
const cubo = new THREE.Mesh(geometry, material); // Objeto 3D
scene.add(cubo); // Agregar a la escena

// Agregamos la luz
const light = new THREE.PointLight(0xffffff, 1); // Luz blanca
light.position.set(5,5,5); // Posición de la luz
scene.add(light); // Añadir a la escena

// Posicionamos la camara
camera.position.z = 5; // Alejar cámara

// Establecemos los controles por teclado
window.addEventListener('keydown', (event) => {

    switch(event.key) {

        // Controles para la traslación (posicion)
        case 'ArrowUp':
            cubo.position.y += 0.2; // mover arriba
            break;

        case 'ArrowDown':
            cubo.position.y -= 0.2; // mover abajo
            break;
```

```
        case 'ArrowLeft':
            cubo.position.x -= 0.2; // mover izquierda
            break;

        case 'ArrowRight':
            cubo.position.x += 0.2; // mover derecha
            break;

        // Controles para la rotacion (orientacion)
        case 'q':
        case 'Q':
            cubo.rotation.y -= 0.1; // rotar izquierda
            break;

        case 'e':
        case 'E':
            cubo.rotation.y += 0.1; // rotar derecha
            break;

        // Controles para el escalado (tamaño)
        case 'z':
        case 'Z':
            cubo.scale.x += 0.1; // aumentar tamaño X
            cubo.scale.y += 0.1; // aumentar tamaño Y
            cubo.scale.z += 0.1; // aumentar tamaño Z
            break;

        case 'x':
        case 'X':
            // Reducir tamaño pero con límite mínimo
            cubo.scale.x = Math.max(0.2, cubo.scale.x - 0.1);
            cubo.scale.y = Math.max(0.2, cubo.scale.y - 0.1);
            cubo.scale.z = Math.max(0.2, cubo.scale.z - 0.1);
            break;
    }

});

// Creamos la funcion para la animacion
function animate() {

    requestAnimationFrame(animate); // Loop infinito

    renderer.render(scene, camera); // Dibujar escena

}

// Ejecutar animación
```

```
animate();

// Ajustes responsive
window.addEventListener('resize', () => {
    camera.aspect = window.innerWidth / window.innerHeight; // nueva
proporción
    camera.updateProjectionMatrix(); // actualizar cámara
    renderer.setSize(window.innerWidth, window.innerHeight); // ajustar
tamaño
});

</script>

</body>
</html>
```